How this demo works

This app is a retrieval-augmented search engine for PDFs. You upload a PDF from the Streamlit frontend, which runs on an AWS EC2 instance behind a Caddy reverse proxy that provides HTTPS.

The frontend uploads the file to a private Amazon S3 bucket under a folder named after your browser session, then sends an ingest event to Inngest. Inngest is a workflow orchestration service: it invokes the backend, retries failed steps, and records every run.

The backend is a FastAPI application packaged as a Docker image and deployed to AWS Lambda. The ingest workflow downloads the PDF, deletes it from S3 immediately, extracts the text with pypdf, and splits it into overlapping chunks of about one thousand characters.

Each chunk is turned into a 384-dimensional embedding by the bge-small-en-v1.5 model, which runs locally inside Lambda through the fastembed library. No paid API key is used anywhere in the pipeline. The embeddings are stored in a Qdrant vector database tagged with your session id.

When you ask a question, the query workflow embeds your question with the same model and asks Qdrant for the closest chunks, filtered to your session only. Other visitors can never see your documents. The best matching passages are returned with a cosine similarity score.

Privacy: uploaded files are deleted from S3 as soon as they are processed, and an Inngest cron job runs every fifteen minutes to delete any vectors older than one hour. An S3 lifecycle rule acts as a backstop and expires anything left in the uploads folder after one day.

Security: the EC2 instance uses an IAM role instead of stored access keys, Streamlit only listens on localhost, Lambda concurrency is capped, and uploads are limited to ten megabytes and fifty pages.

Try asking: Where is my PDF stored? What embedding model is used? How long is my data kept? What does Inngest do?